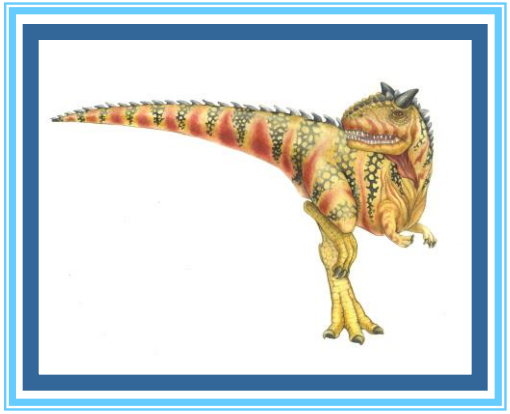
Chapter 9: Virtual Memory



Silberschatz, Galvin and Gagne ©2018



Objectives

- Define virtual memory and describe its benefits.
- Illustrate how pages are loaded into memory using demand paging.
- Apply the FIFO, optimal, and LRU page-replacement algorithms.
- Describe the working set of a process, and explain how it is related to program locality.





Background

- Code needs to be in memory to execute, but entire program rarely used
  - Error code, unusual routines, large data structures
- Entire program code not needed at same time
- Consider ability to execute partially-loaded program
  - Program no longer constrained by limits of physical memory
  - Each program takes less memory while running -> more programs run at the same time
    - Increased CPU utilization and throughput with no increase in response time or turnaround time
  - Less I/O needed to load or swap programs into memory -> each user program runs faster





Virtual memory

- **Virtual memory** – separation of user logical memory from physical memory
  - Only part of the program needs to be in memory for execution
  - Logical address space can therefore be much larger than physical address space
  - Allows address spaces to be shared by several processes
  - Allows for more efficient process creation
  - More programs running concurrently
  - Less I/O needed to load or swap processes





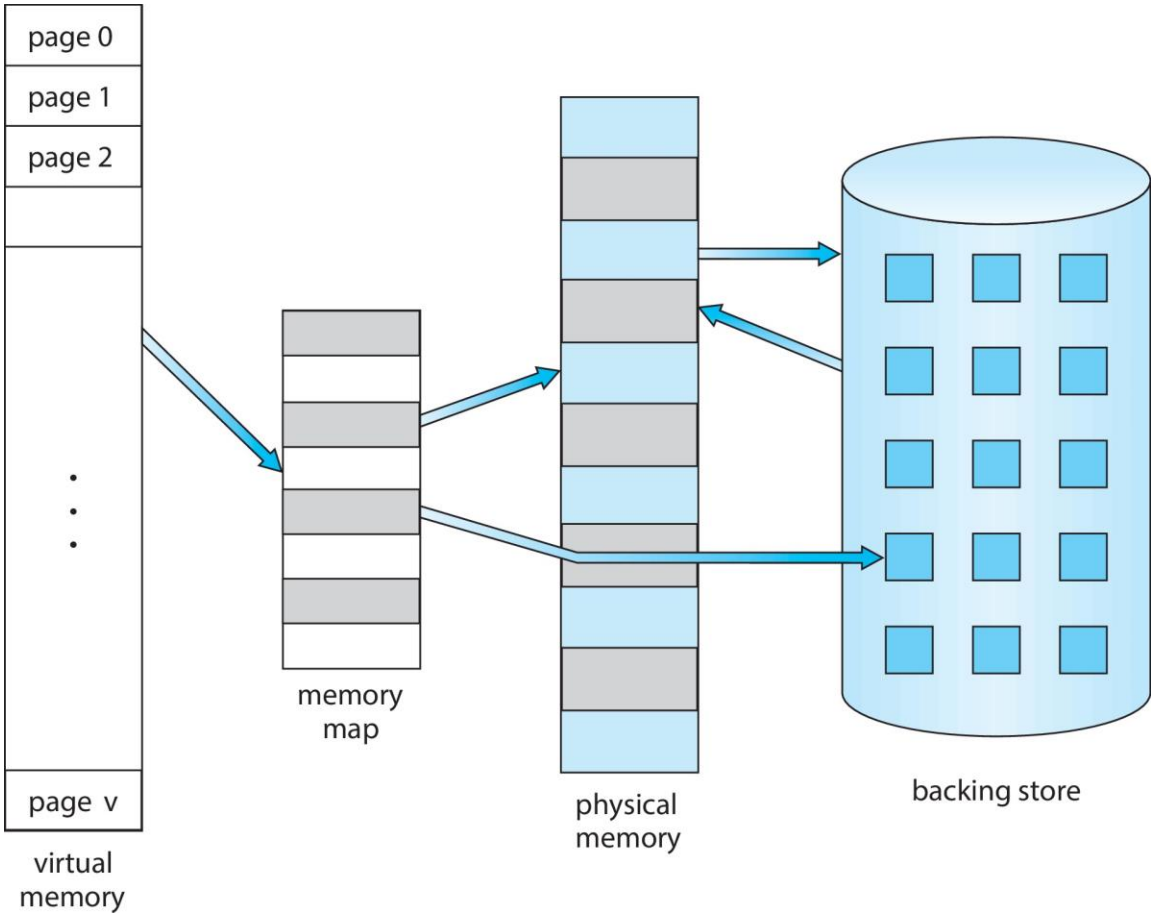
Virtual memory (Cont.)

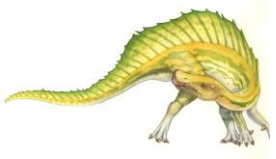
- **Virtual address space** – logical view of how process is stored in memory
  - Usually start at address 0, contiguous addresses until end of space
  - Meanwhile, physical memory organized in page frames
  - MMU must map logical to physical
- **Virtual memory can be implemented via:**
  - Demand paging
  - Demand segmentation





Virtual Memory That is Larger Than Physical Memory

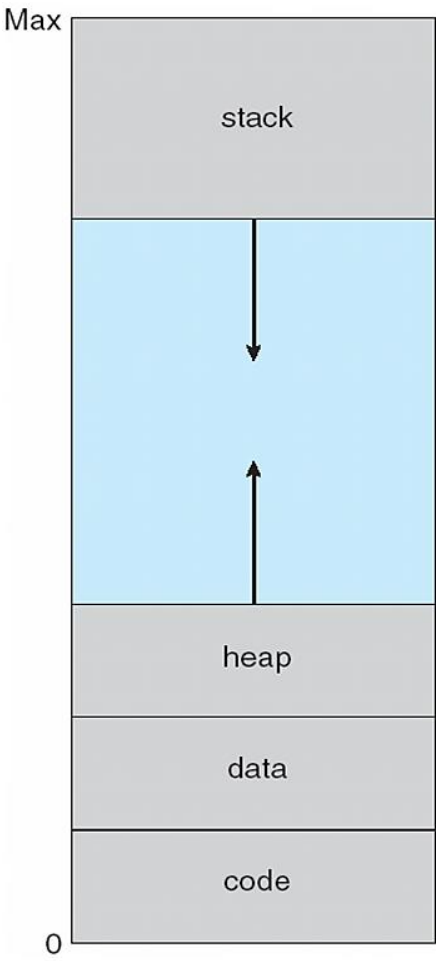


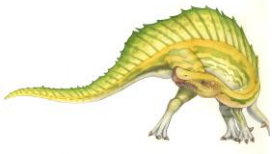




Virtual-address Space

- Usually design logical address space for stack to start at Max logical address and grow “down” while heap grows “up”
  - Maximizes address space use.
  - The large blank space (or hole) between the heap and the stack is part of the virtual address space but will require actual physical pages only if the heap or stack grows.
- Enables **sparse** address spaces with holes left for growth, dynamically linked libraries, etc.







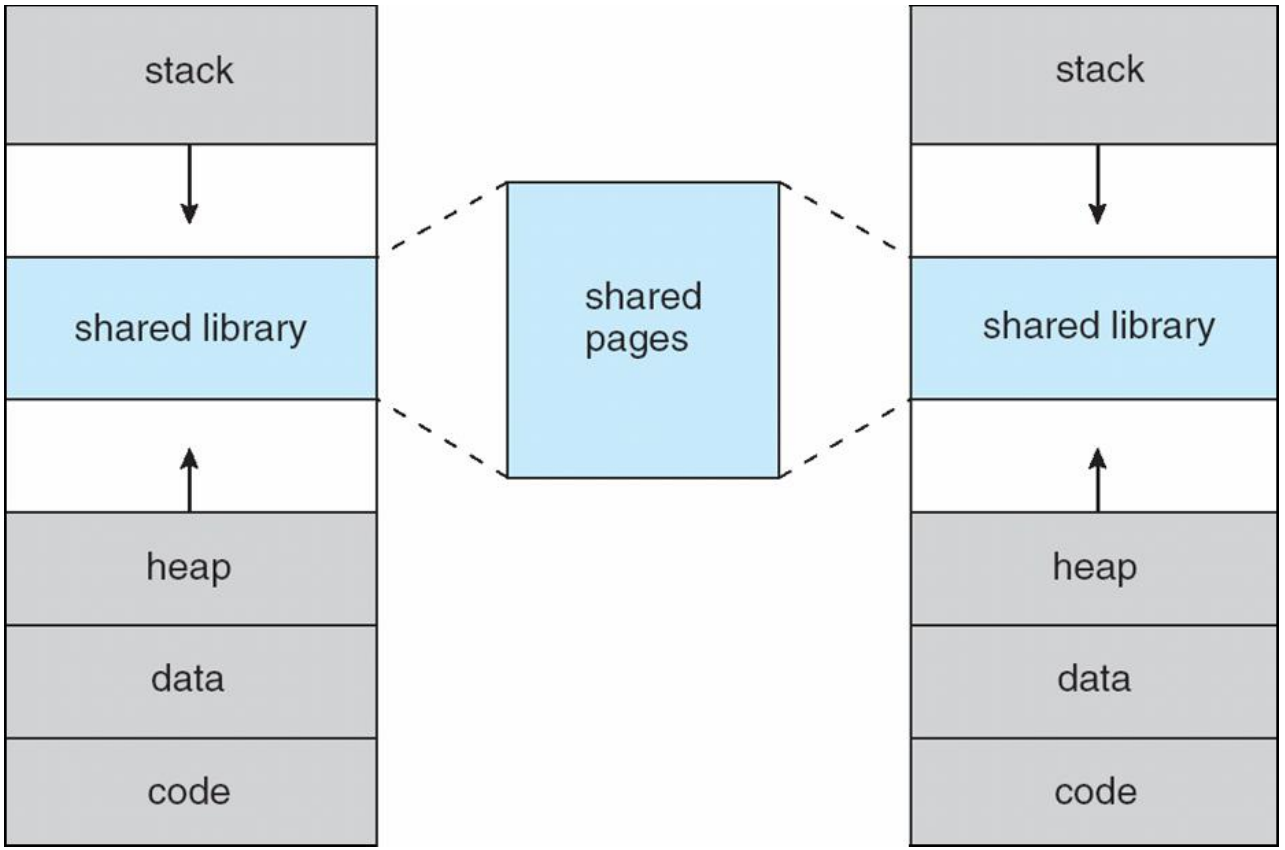
Virtual-address Space

- **Virtual memory** allows files and memory to be shared by two or more processes through page sharing.
- **This leads to the following benefits:**
  - System libraries can be shared by several processes via mapping into virtual address space.
  - Similarly, processes can share memory. Virtual memory allows one process to create a region of memory that it can share with another process.
  - Pages can be shared during process creation with the **fork ( )** system call, thus speeding up process creation.





Shared Library Using Virtual Memory







Demand Paging

- Consider how an executable program might be loaded from disk into memory.
- One option is to load the entire program in physical memory at program execution time.
- **The problem with this approach** is that we may not initially need the entire program in memory.
- Suppose a program starts with a list of available options from which the user is to select. Loading the entire program into memory results in loading the executable code for all options, regardless of whether or not an option is ultimately selected by the user.





Demand Paging

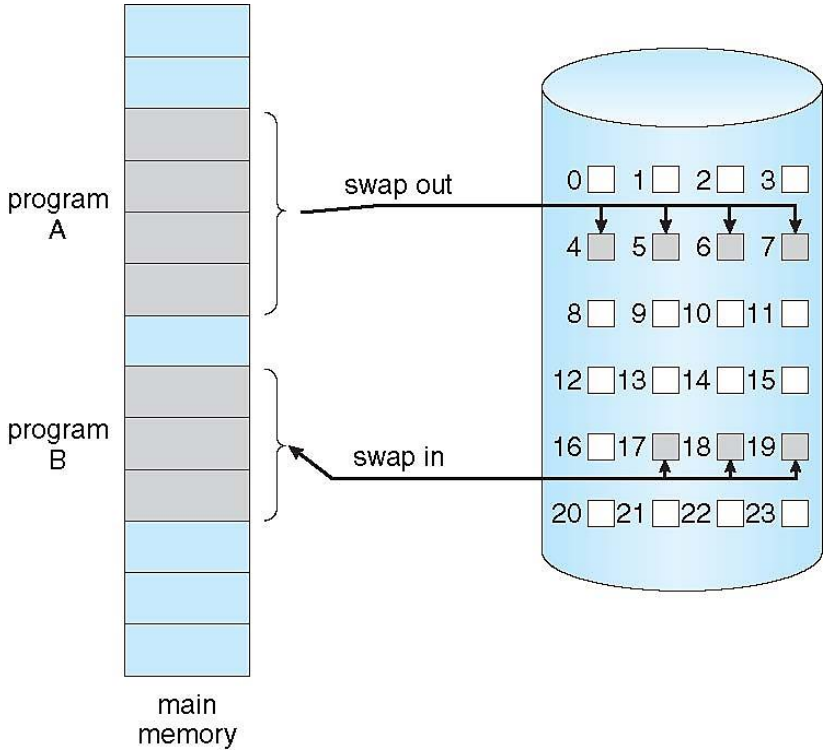
- An alternative strategy is to load pages only as they are needed.
- This technique is known as **demand paging** and is commonly used in **virtual memory systems**.
- With demand-paged virtual memory, pages are loaded only when they are demanded during program execution. Pages that are never accessed are thus never loaded into physical memory.

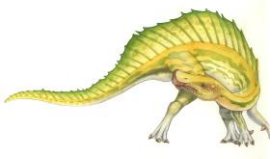




Demand Paging

- Could bring entire process into memory at load time or bring a page into memory only when it is needed.
  - Less I/O needed, no unnecessary I/O
  - Less memory needed
  - Faster response
  - More users
- A demand-paging system is similar to a paging system with swapping where processes reside in secondary memory (usually a disk).



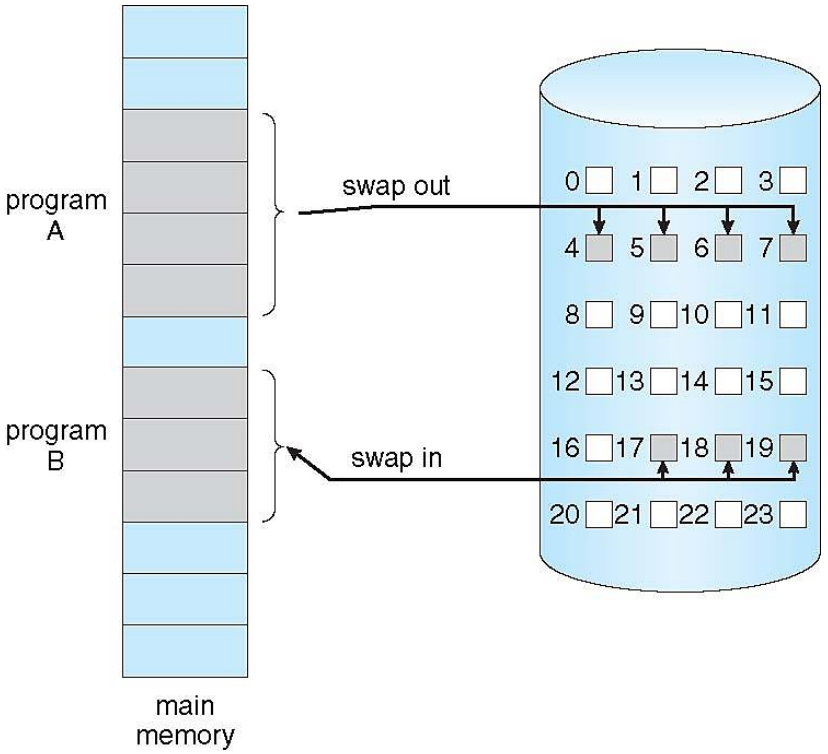


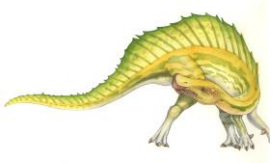
Silberschatz, Galvin and Gagne ©2018



Demand Paging

- When we want to execute a process, we swap it into memory. Rather than swapping the entire process into memory, though, we use **a lazy swapper**.
- **Lazy swapper** – never swaps a page into memory unless page will be needed
- Swapper that deals with pages is a **pager**







Basic Concepts

- With swapping, pager guesses which pages will be used before swapping out again.
- Instead, pager brings in only those pages into memory.
- If pages needed are already **memory resident:**
  - No difference from non demand-paging
- If page needed and **not memory resident**:
  - Need to detect and load the page into memory from storage.
    - Without changing program behavior.
    - Without programmer needing to change code.





Basic Concepts

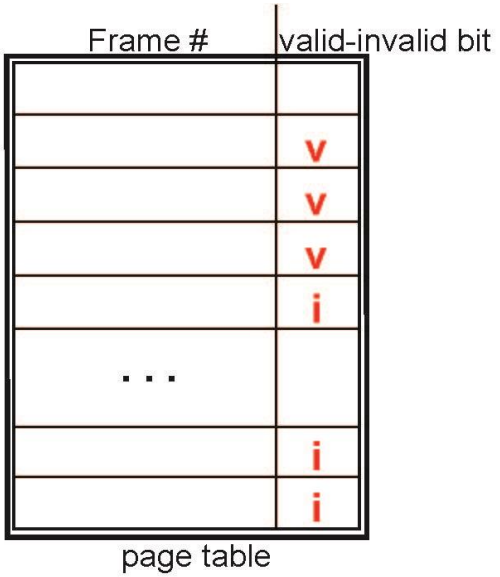
- With this scheme, we need some form of hardware support to distinguish between the pages that are in memory and the pages that are on the disk (out of memory).
- **The valid–invalid bit scheme** can be used for this purpose.
- This time, however, when this bit is set to “valid,” the associated page is both legal and in memory. If the bit is set to “invalid,” the page either is not valid (that is, not in the logical address space of the process) or is valid but is currently on the disk.





Valid-Invalid Bit

- With each page table entry a valid–invalid bit is associated (**v** ⇒ in-memory – **memory resident**, **i** ⇒ not-in-memory)
- Initially valid–invalid bit is set to **i** on all entries
- **Example of a page table snapshot:**

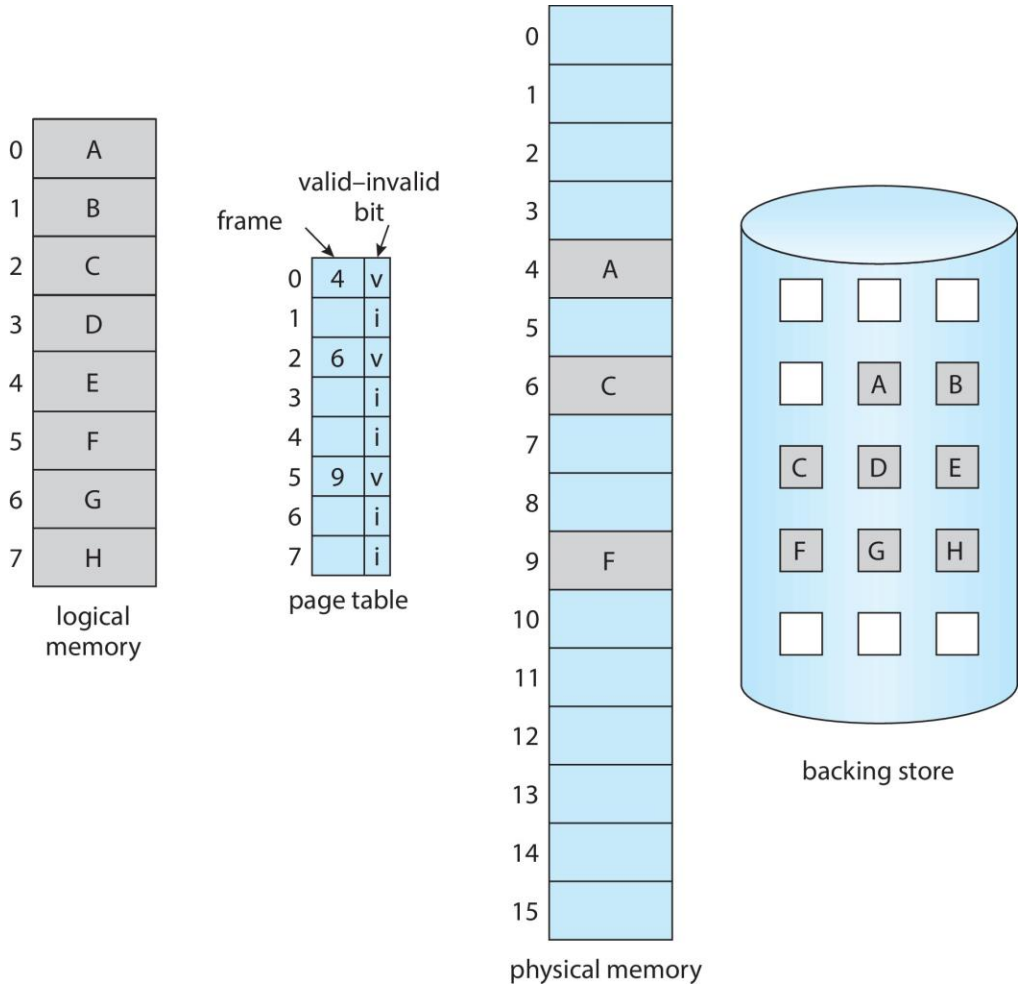


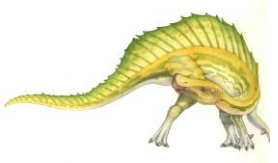
- During MMU address translation, if valid–invalid bit in page table entry is **i** ⇒ page fault.





Page Table When Some Pages Are Not in Main Memory





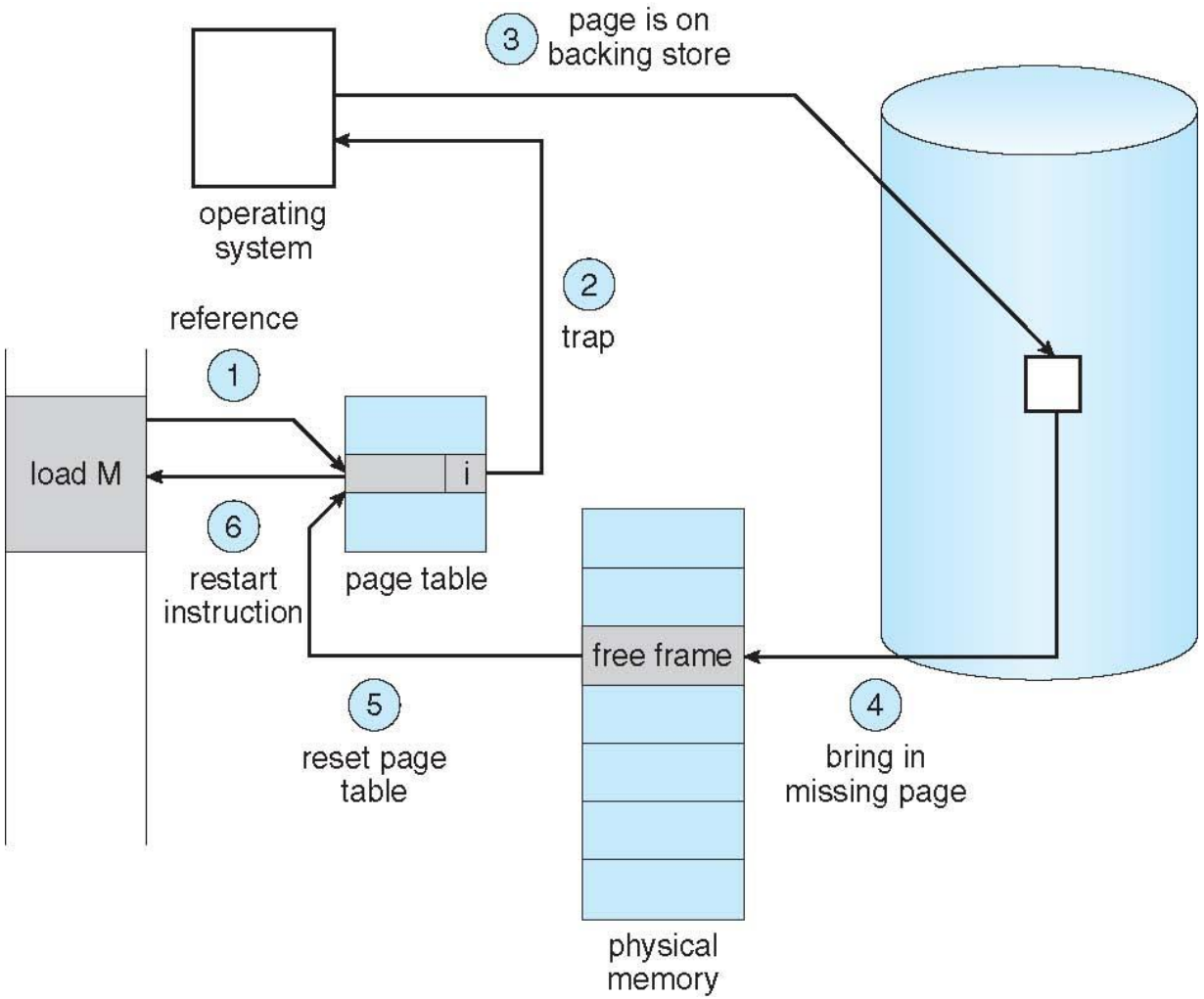


Steps in Handling Page Fault

1. If there is a reference to a page, first reference to that page will trap to operating system
   - Page fault
2. Operating system looks at another table to decide:
   - Invalid reference $\Rightarrow$ abort
   - Just not in memory
3. Find free frame
4. Swap page into frame via scheduled disk operation
5. Reset tables to indicate page now in memory
   Set validation bit = **v**
6. Restart the instruction that caused the page fault



Steps in Handling a Page Fault (Cont.)





Aspects of Demand Paging

- In the **extreme case**, we can start executing a process with no pages in memory.
- OS sets instruction pointer to first instruction of process, non-memory-resident -> page fault
- After this page is brought into memory, the process continues to execute, faulting as necessary until every page that it needs is in memory. At that point, it can execute with no more faults.
- This scheme is **pure demand paging**: never bring a page into memory until it is required.





Aspects of Demand Paging

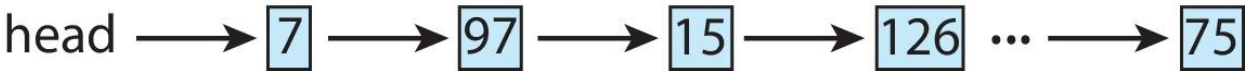
- Actually, a given instruction could access multiple pages → multiple page faults.
- This situation would result in unacceptable system performance.
- **Hardware support needed for demand paging:**
  - Page table with valid / invalid bit
  - Secondary memory
  - Instruction restart





Free-Frame List

- When a page fault occurs, the operating system must bring the desired page from secondary storage into main memory.
- Most operating systems maintain a **free-frame list** -- a pool of free frames for satisfying such requests.



- Operating system typically allocate free frames using a technique known as **zero-fill-on-demand** -- the content of the frames zeroed-out before being allocated.
- When a system starts up, all available memory is placed on the free-frame list.





Copy-on-Write

- **Copy-on-Write** (COW) is a technique allows both parent and child processes to initially ***share*** the same pages in memory.
  - If either process modifies a shared page, only then is the page copied.
- **Example:**
  - Assume that the child process attempts to modify a page containing portions of the stack, with the pages set to be copy-on-write.
  - The operating system will create a copy of this page, mapping it to the address space of the child process.
  - The child process will then modify its copied page and not the page belonging to the parent process.





Copy-on-Write

- Obviously, when the copy-on-write technique is used, only the pages that are modified by either process are copied.
- Only pages that can be modified need be marked as copy-on-write.
- Pages that cannot be modified (pages containing executable code) can be shared by the parent and child.
- **Copy-on-write** is a common technique used by several operating systems, including Windows XP, Linux, and Solaris.





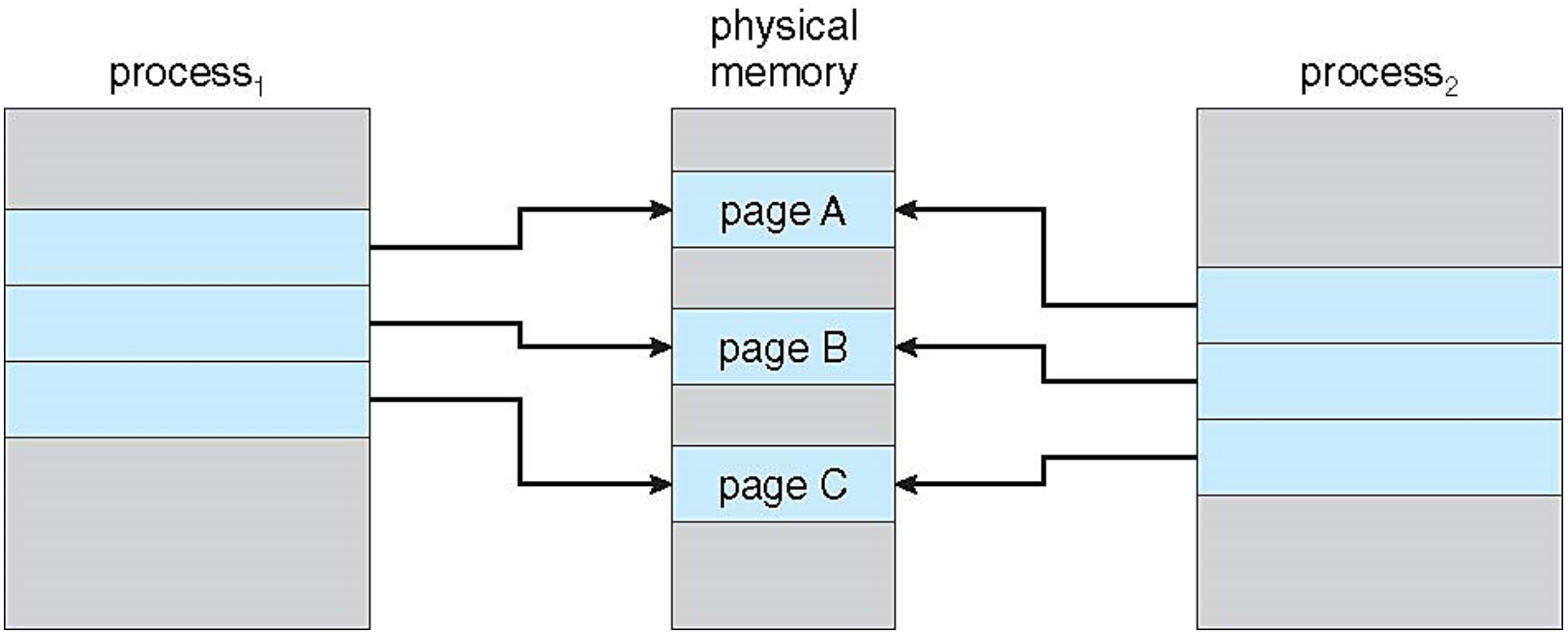
Copy-on-Write

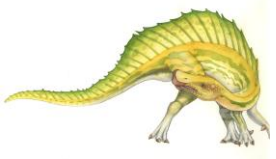
- In general, free pages are allocated from a **pool** of **zero-fill-on-demand** pages
  - Pool should always have free frames for fast demand page execution
    - Don’t want to have to free a frame as well as other processing on page fault
  - Why zero-out a page before allocating it?
- `vfork()` variation on `fork()` system call has parent suspend and child using copy-on-write address space of parent
  - Designed to have child call `exec()`
  - Very efficient





Before Process 1 Modifies Page C

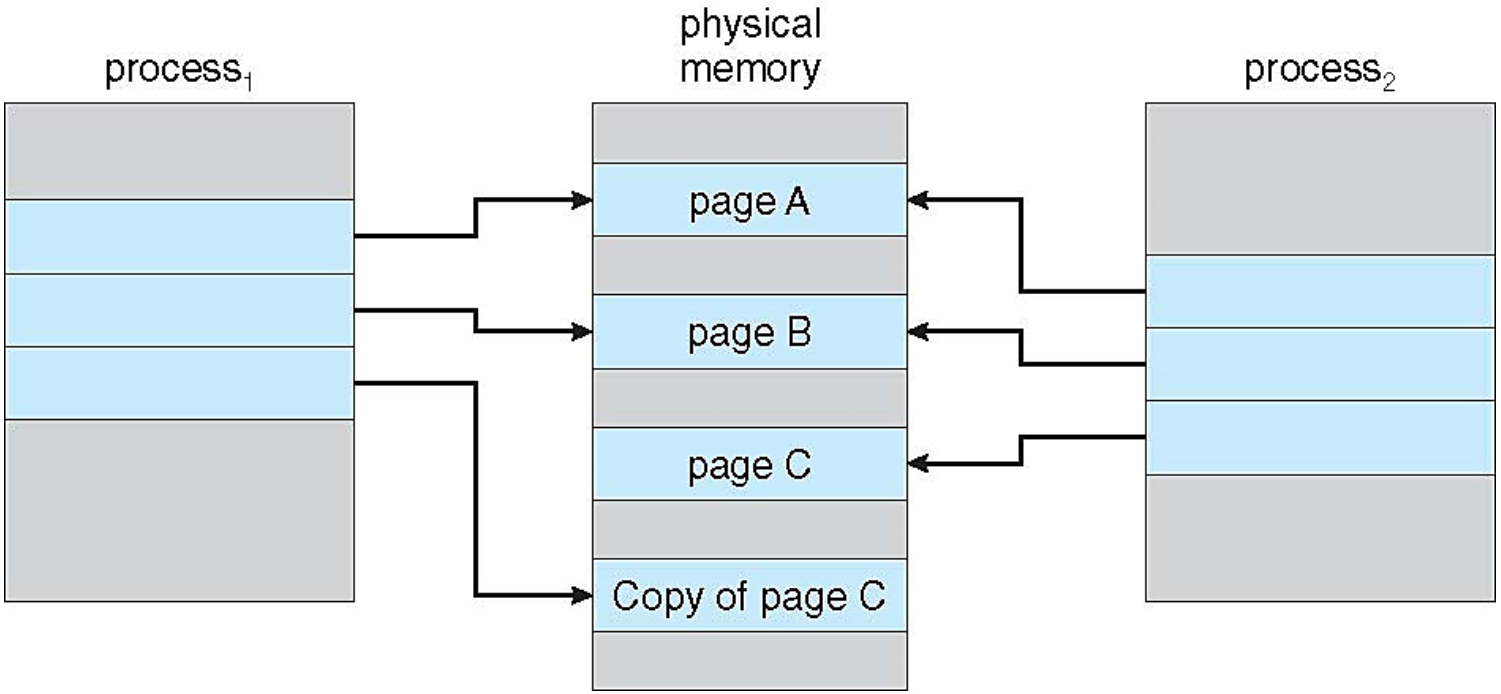




Silberschatz, Galvin and Gagne ©2018



After Process 1 Modifies Page C





Silberschatz, Galvin and Gagne ©2018



What Happens if There is no Free Frame?

- Used up by process pages
- Also in demand from the kernel, I/O buffers, etc
- How much to allocate to each?
- Page replacement – find some page in memory, but not really in use, page it out
  - Algorithm – terminate? swap out? replace the page?
  - Performance – want an algorithm which will result in minimum number of page faults
- Same page may be brought into memory several times



Silberschatz, Galvin and Gagne ©2018



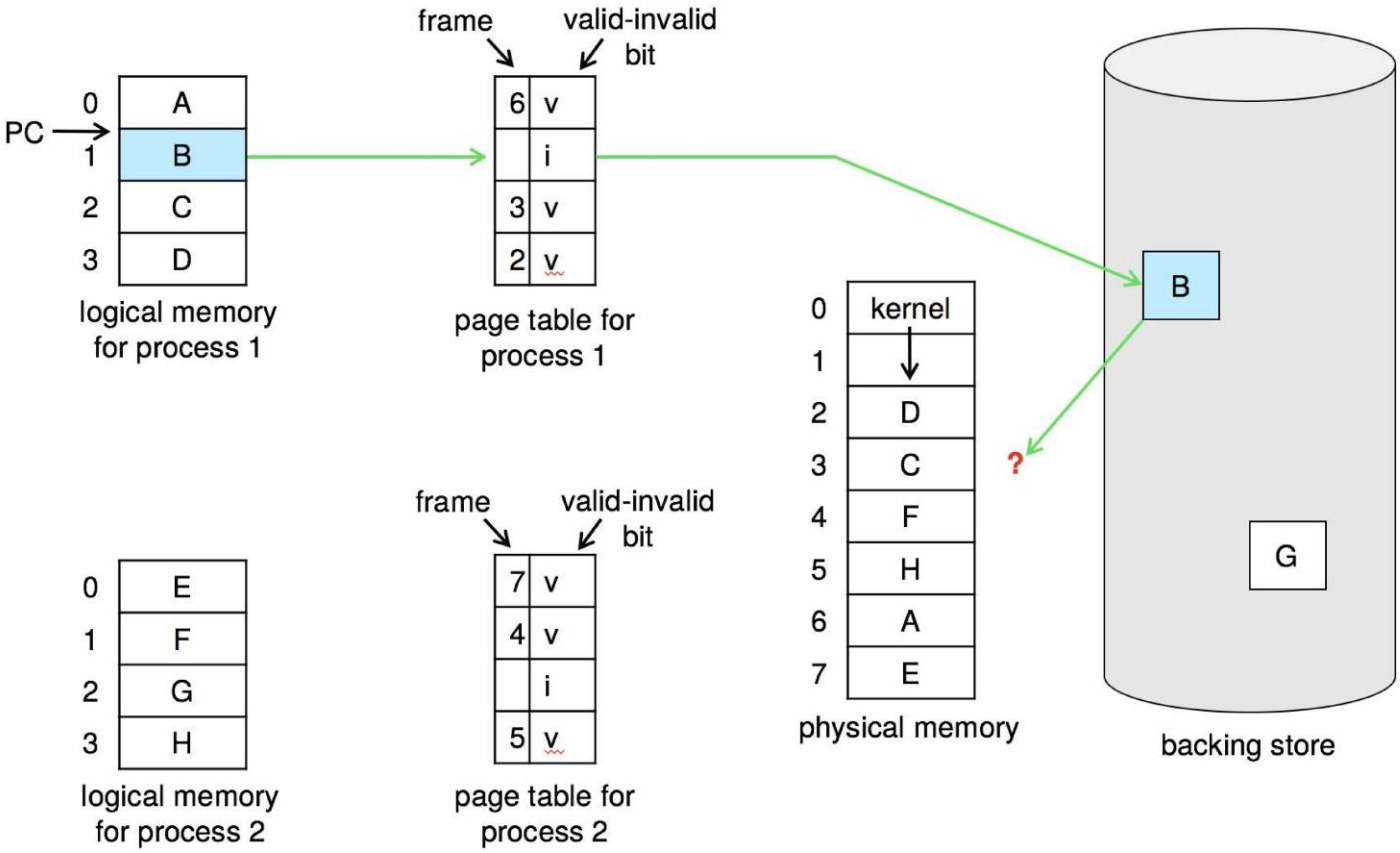
Page Replacement

- **Page replacement** refers to the process of deciding which pages to remove from physical RAM when new pages need to be loaded but there is insufficient space.
- **Over-allocating** typically means that the system assigns more virtual memory to a process than can physically fit in the main memory (RAM).
- This can lead to page faults, where the operating system must load a page from disk into memory, and eventually, page replacement algorithms come into play.





Need For Page Replacement







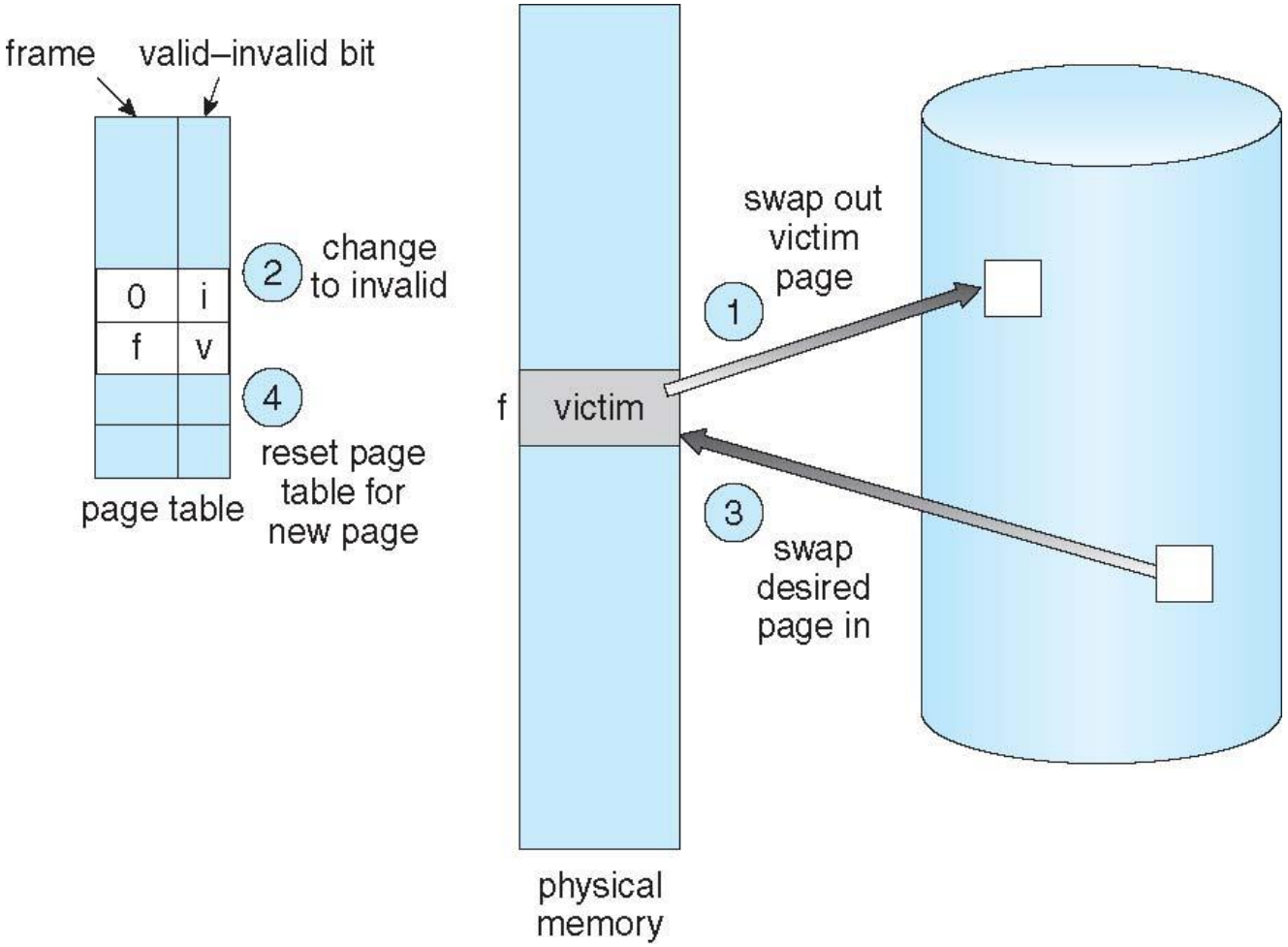
Basic Page Replacement

1. Find the location of the desired page on disk
2. Find a free frame:
   - If there is a free frame, use it
   - If there is no free frame, use a page replacement algorithm to select a **victim frame.**
   - Write victim frame to disk if dirty
3. Bring the desired page into the (newly) free frame; update the page and frame tables
4. Continue the process by restarting the instruction that caused the trap





Page Replacement







Page Replacement

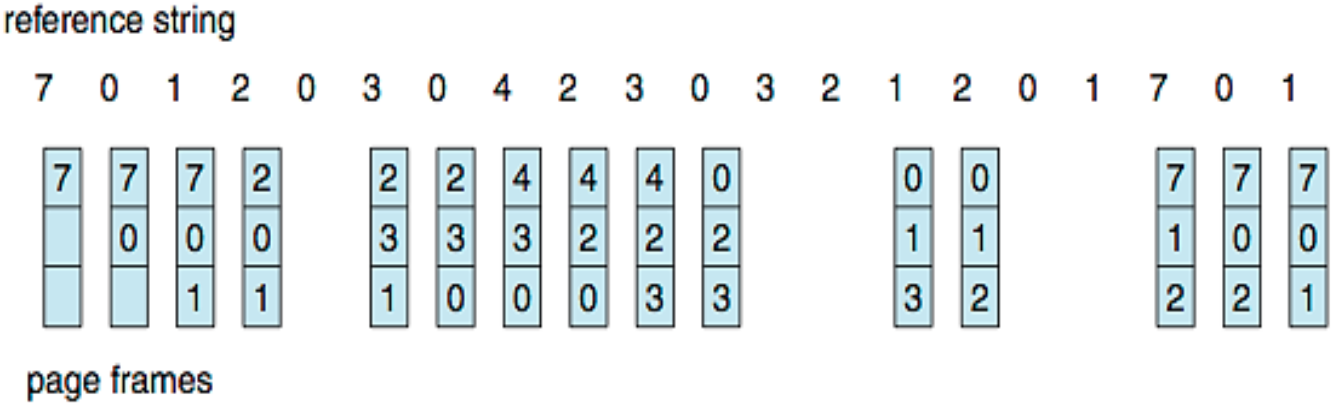
- Once the memory is full and a new page is needed, the page replacement algorithm decides which page in memory should be swapped out to make room for the new page.
- **Common page replacement algorithms:**
- **FIFO (First-In, First-Out):** The oldest page in memory is swapped out.
- **LRU (Least Recently Used):** The least recently accessed page is swapped out.
- **Optimal Page Replacement:** The page that will not be used for the longest time in the future is swapped out.





First-In-First-Out (FIFO) Algorithm

- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)



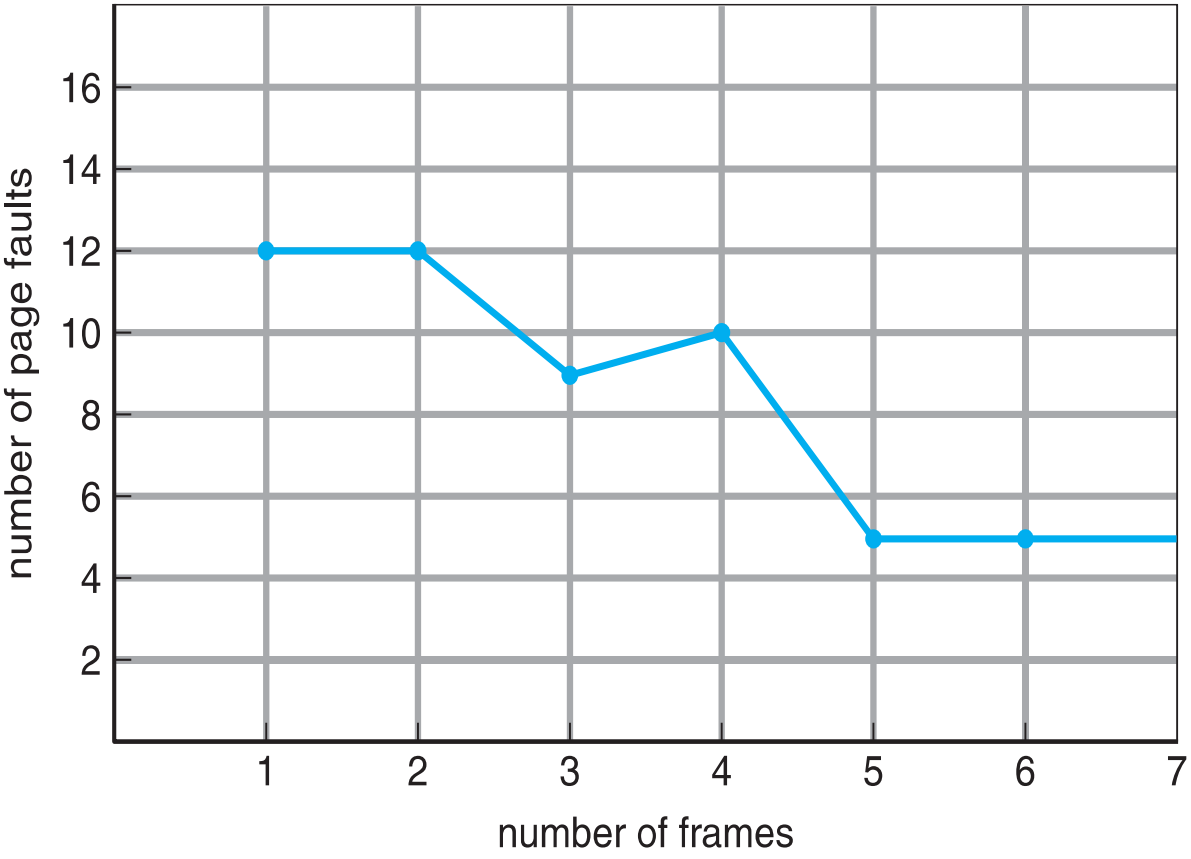
15 page faults

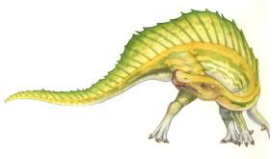
- Can vary by reference string: consider 1,2,3,4,1,2,5,1,2,3,4,5
  - Adding more frames can cause more page faults!
    - **Belady's Anomaly**





FIFO Illustrating Belady's Anomaly



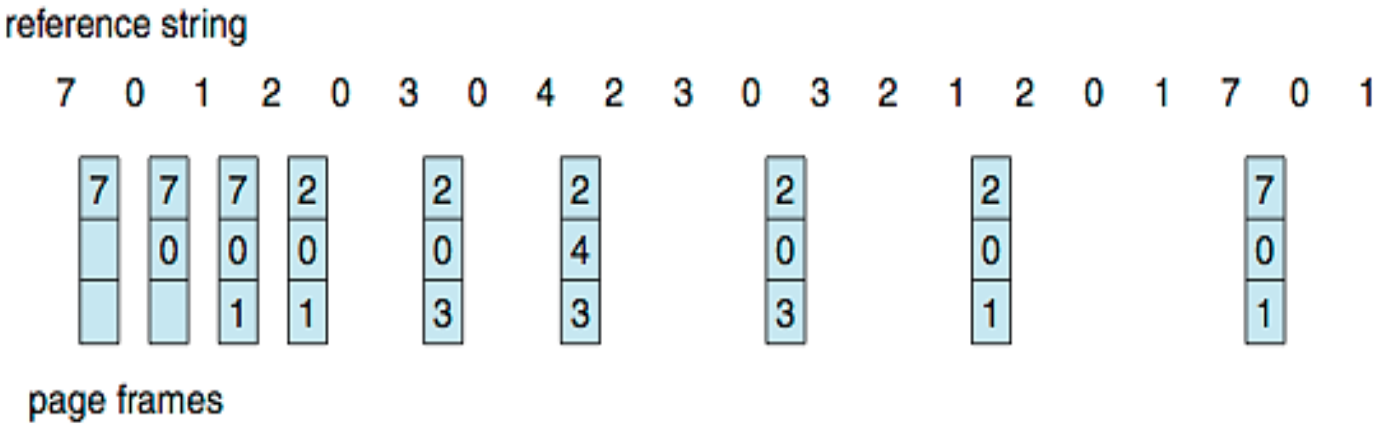


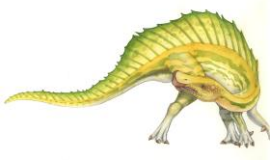
Silberschatz, Galvin and Gagne ©2018



Optimal Algorithm

- Replace page that will not be used for longest period of time
  - 9 page faults is optimal for the example
- How do you know this?
  - Can't read the future
- Used for measuring how well your algorithm performs

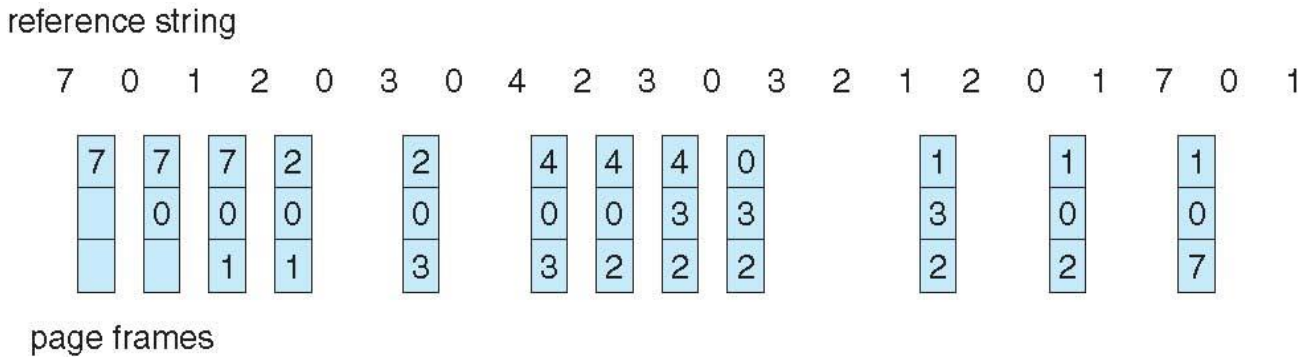






Least Recently Used (LRU) Algorithm

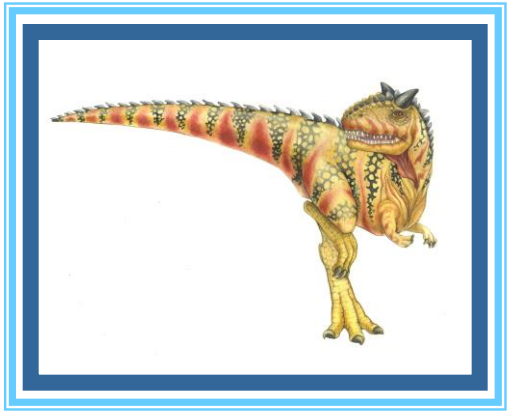
- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page



- 12 page faults – better than **FIFO** but worse than **OPT**
- Generally good algorithm and frequently used



End of Chapter 9



Silberschatz, Galvin and Gagne ©2018